

Joint Low-Discrepancy Sampling for Reduced-Parameter Arrhenius Uncertainty Quantification: Sobol and Latin Hypercube Alternatives to Pseudo-Random Design Matrices

Krunal R. Panchal

Thermodynamics and Combustion Laboratory, Department of Mechanical Engineering, IIT Madras

June 16, 2026

Abstract

The MUQ-SAC / mechanism-optimization-suite design-matrix generator (`DesignMatrix.py`, `Uncertainty.py`) originally populated its polynomial-response-surface (PRS) training matrix using independent calls to `numpy.random`, both for the full-parameter (A1+B1+C1) design and for the partial-parameter (p-SAC) reduced-Cholesky design. This note documents a drop-in replacement that draws the same matrix from a *joint* low-discrepancy sequence – scrambled Sobol’ or Latin Hypercube Sampling (LHS) – spanning every reaction simultaneously, while leaving every downstream consumer (Class-A/B/C curve construction, the four `_psac_*` rejection-sampling routines, and the random-shuffling step) untouched in its numerical behaviour. The central engineering difficulty is that several of the partial-parameter routines use rejection sampling, which consumes a variable number of draws per accepted sample; a naively shared low-discrepancy stream would have every rejection desynchronise all subsequent samples’ coordinates, destroying exactly the joint structure the switch was meant to provide. We resolve this by reserving one row of a pre-generated low-discrepancy block per *accepted output sample* (consumed only on that sample’s first attempt) and routing any retry to an independent, deterministically seeded fallback stream. We report the resulting discrepancy improvements, the measured fallback/overflow rates of the rejection-sampling routines under this scheme, and the discrepancy cost of the existing random-shuffling step, which is shown to be substantial.

Contents

1	Motivation	2
2	Background: discrepancy and the two candidate methods	2
3	Implementation	3
3.1	Joint sampling across reactions	3
3.2	The rejection-sampling problem and its resolution	4
3.3	A known, accepted edge case: f-SAC at $m = 1$	4
3.4	Reproducibility	5
3.5	The degenerate-case padding RNGs: an explicit scope boundary	5
3.6	Cache invalidation	5

4 Empirical results	5
4.1 Discrepancy improvement	5
4.2 Fallback/overflow rates under rejection sampling	6
4.3 The random-shuffling step erodes joint structure	6
5 Practical recommendations	7
6 Scope explicitly not covered by this pass	7
7 Conclusion	7

1 Motivation

The design matrix produced by `DesignMatrix.py` exists to fit polynomial response surfaces (PRS) over the Cholesky-parameterised Arrhenius uncertainty space of each reaction. The statistical quality of a PRS fit at a fixed sample budget N is governed less by N itself than by how evenly the N design points cover the input hypercube: clustered or gapped designs waste samples on redundant regions while leaving other regions of the uncertainty box unprobed, inflating the variance of the fitted polynomial coefficients and, downstream, the variance of any sensitivity index or optimum computed from the surrogate. This is precisely the distinction between *pseudo-random* sampling (i.i.d. uniform draws, which cover space with $O(N^{-1/2})$ integration error and visible small- N clustering) and *low-discrepancy* sampling (deterministic or quasi-random sequences engineered to cover space as evenly as possible, with $O(N^{-1})$ to $O(N^{-1}(\log N)^d)$ error under the Koksma–Hlawka inequality).

The original generator drew every Class-A, Class-B, and Class-C curve – in both the full A1+B1+C1 path (`getSamples`) and the reduced-parameter p-SAC path (`getSample_partial`) – from independent `np.random.random_sample` calls, one reaction at a time. Two changes were requested and are implemented here:

1. Replace the per-reaction independent draws with a *single joint* low-discrepancy draw spanning every reaction’s allocated columns at once, so that the low-discrepancy guarantees apply to the full joint design, not just to each reaction’s own one-dimensional marginal.
2. Make the sampling method configurable (`'numpy' / 'sobol' / 'lhs'`), defaulting to scrambled Sobol’, with Latin Hypercube as a second low-discrepancy option and the original pseudo-random behaviour retained for direct comparison.

2 Background: discrepancy and the two candidate methods

For a point set $P = \{x_1, \dots, x_N\} \subset [0, 1)^d$, the star discrepancy $D^*(P)$ measures the worst-case deviation between the empirical fraction of points in an axis-aligned box $[0, b)$ and the box’s volume b . The Koksma–Hlawka inequality bounds the integration error of a function f of bounded variation $V(f)$ by $|\hat{I} - I| \leq V(f) D^*(P)$, so any reduction in discrepancy is a direct, function-independent bound on integration/fitting error. Two practical surrogates for star discrepancy are used here: the centred L_2 discrepancy (CD) and the wrap-around discrepancy (WD), both computed via `scipy.stats.qmc.discrepancy`.

Scrambled Sobol’ sequences are digital (t, m, d) -nets: deterministic low-discrepancy sequences whose discrepancy guarantees are derived for sample sizes that are exact powers of two ($N = 2^m$). Scrambling (Owen-style, the `scipy` default when `scramble=True`) randomises the sequence while preserving its equidistribution properties, which is what permits a useful notion of “seed” and turns the otherwise-deterministic sequence into something that can be reproducibly re-randomised. *This is also why the implementation below rounds every requested*

sample count up to the next power of two for this method rather than truncating: truncating a Sobol' sequence to a non-power-of-two prefix degrades exactly the balance property that motivates using it at all.

Latin Hypercube Sampling (LHS) stratifies each one-dimensional marginal into N equal-probability bins and guarantees exactly one sample per bin per dimension, with the cross-dimension pairing then optimised (here, `optimization='random-cd'`, which greedily improves centred L_2 discrepancy over a fixed number of random column permutations) to also improve joint coverage. Unlike Sobol', LHS imposes no power-of-two constraint and is used here at the exact requested sample count.

3 Implementation

3.1 Joint sampling across reactions

For both the full and partial paths, every reaction's "generator" array (whatever shape it had under the legacy `np.random.random_sample` call) is now a column-slice of one matrix produced by a single call to the new `qmc_sampling.joint_uniform_block` (full path) or `qmc_sampling.build_joint_shadow` (partial path, see §3.2):

```
def joint_uniform_block(method, n_required, dim, seed=None,
                       optimization="random-cd"):
    if method == "sobol":
        n_actual = ceil_pow2(n_required)
        engine = qmc.Sobol(d=dim, scramble=True, seed=seed)
        return engine.random_base2(n_actual.bit_length() - 1), n_actual
    if method == "lhs":
        engine = qmc.LatinHypercube(d=dim, seed=seed,
                                    optimization=optimization)
        return engine.random(n=n_required), n_required
    if method == "numpy":
        rng = np.random.default_rng(seed)
        return rng.random((n_required, dim)), n_required
```

Reaction j 's columns are then `block[:, j*dc:(j+1)*dc]` for a class with d_c raw values per reaction per sample. The per-class dimensionality budget used throughout is summarised in Table 1.

Table 1: Raw values consumed per reaction per accepted sample, by class.

Class	d_c	Consumers
A (full)	1	direct scalar multiplying $\zeta_{\max}$; no rejection.
A (partial)	2	$\alpha_s \sim U(0, 1)$, $\text{sign} \in \{-1, +1\}$ in <code>_psac_class.A</code> .
B (full)	2	$(a_1, a_2) \rightarrow \text{populateValues} \rightarrow \text{SHGO solve getB2Zeta}$; no rejection.
B (partial)	4	$(a_1, a_2) \rightarrow \text{populateValues}$ (meaningful only for $m = 3$, which defers to <code>getB2Zeta</code>); (r_1, r_2) reserved for the $m < 3$ analytical bisection's first attempt.
C (full)	2	$(a_1, a_2) \rightarrow \text{populateValues} \rightarrow \text{getC2Zeta}$; no rejection.
C (partial)	2	(r_1, r_2) first-attempt draw for <code>_psac_class.C</code> .
f-SAC (partial)	3	(r_1, r_2) -factor, r_{mid} -factor) first-attempt draw for <code>_psac_fsac</code> .

All values are stored and consumed in their natural $[0,1)$ form; every existing call site already performs its own affine rescaling (e.g. `rng.uniform(-1,1)`, `rng.uniform(0.3,1.0)`), so no additional rescaling is introduced – this also means a single row column can transparently feed call sites that request different ranges without any bookkeeping for “which range was this drawn for.”

3.2 The rejection-sampling problem and its resolution

Four routines in `Uncertainty.py` – `_psac_class_A`, `_psac_class_D.m2` (Class-B/D, $m = 2$), `_psac_class_C`, and `_psac_fsac` – generate each accepted output sample via rejection sampling: draw, evaluate a feasibility/sign-change criterion, and either accept or redraw, up to a bounded number of attempts. If such a routine consumed a flat low-discrepancy sequence call-by-call, a single rejected attempt for sample i would shift every later sample’s coordinates by however many extra draws that rejection cost, so sample $i + 1$ would see whatever coordinates the sequence happened to assign next rather than its own deliberately-placed point. The joint, designed structure the whole exercise was meant to provide would be destroyed by the very first rejection.

The resolution implemented in `qmc_sampling.QuasiRandomGenerator` is to decouple *rows* from *calls*: one row of the shadow block is reserved per desired accepted sample, consumed only on that sample’s first attempt; any draw beyond the row’s d_c values (i.e. any retry) is transparently routed to an independent, deterministically seeded `np.random.default_rng` fallback stream instead. The object exposes the same `.uniform()/choice()/permutation()/random()` surface as `np.random.Generator`, so it is a drop-in replacement at every existing call site; the only addition is a single `advance_sample()` call inserted at the point in each `_psac.*` routine where bookkeeping should move on to the next reserved row – at the top of each output-sample iteration for the two routines with an explicit per-sample outer loop (`_psac_class_A`, `_psac_class_D.m2`), and once before a shared retry loop plus once after each successful accept for the two routines that share a single retry budget across all requested samples (`_psac_class_C`, `_psac_fsac`).

This is the same shadow-block treatment requested for f-SAC specifically: `get_fSAC_partial` now builds one `QuasiRandomGenerator` per reaction from its 3-column slice of a jointly-drawn `shadow_fsac` block, with a deterministically derived per-reaction fallback seed, exactly mirroring Class-A/B/C-partial.

For the two classes dispatched via multiprocessing (B and C partial, where every worker task only ever produces a single sample), the reservation simplifies: each task already receives its own dedicated shadow row as the `generator` array passed into `run_sampling_b_partial` / `run_sampling_c_partial`; any retry beyond the first attempt is routed to a fallback stream seeded from `derive_seed(base_seed, "classB"/"classC", *generator_values)`, which gives every task an independent fallback stream without needing a separate per-task seed array threaded through the multiprocessing dispatch.

3.3 A known, accepted edge case: f-SAC at $m = 1$

f-SAC is mathematically undefined for a single active parameter and `_psac_fsac` already delegated this case to `_psac_class_A` recursively on the same `rng`. Because `_psac_class_A` performs its own independent `advance_sample()` bookkeeping, the $m = 1$ branch ends up advancing the shared row cursor twice per outer iteration (once for f-SAC’s own three – here unused – draws, once inside the recursive call). Row alignment for $m = 1$ reactions under f-SAC is consequently looser than for $m = 2, 3$. Given $m = 1$ is already a degenerate fallback case, this is accepted as a documented limitation rather than engineered away with additional bookkeeping.

3.4 Reproducibility

An optional master `seed` threads through every entry point. When provided, every per-reaction, per-class fallback stream is derived deterministically via `qmc_sampling.derive_seed(seed, *address)`, where `*address` includes the reaction name and class tag (and, for multiprocessing tasks, the task’s own shadow values, since no separate per-task index is otherwise available across the process boundary). `seed=None` reproduces the original ”no fixed seed” behaviour exactly.

3.5 The degenerate-case padding RNGs: an explicit scope boundary

Reactions with only one active parameter ($m < 2$) have no valid Class-B or Class-C curves; the existing code backfills those slots with extra Class-A draws. This backfill, and the analogous ”pad up to the target row count” safety net, are not part of the primary jointly-sampled design and are intentionally left on a plain, seeded `np.random.default_rng` rather than given the full Sobol/LHS treatment – they are a secondary correction for an already-degenerate case, and extending the joint-shadow machinery to them would add complexity without a clear payoff. This is a deliberate scope boundary, easily revisited if desired.

3.6 Cache invalidation

The on-disk caches for generated curves (`a_type.samples*.pkl`, etc.) are now tagged with the sampling method in their filename. The original cache keys never depended on how the samples were generated, so switching methods between runs would silently reuse another method’s stale cached samples – a genuine bug, now fixed. Cache keys are *not* currently seed-aware (two runs with the same method but different seeds will share a cache); this was not flagged as a problem and is a one-line extension of the same tagging mechanism if full per-seed caching is wanted.

4 Empirical results

4.1 Discrepancy improvement

Table 2 reports the centred L_2 discrepancy of a single joint draw at several sample counts, $d = 2$ (the Class-A-partial / Class-C-partial dimensionality for a 1-reaction slice), all methods seeded identically for a fair comparison. Both low-discrepancy methods are one to two orders of magnitude below pseudo-random at every N tested, and the gap widens with N , consistent with the $O(N^{-1})$ versus $O(N^{-1/2})$ asymptotics.

Table 2: Centred L_2 discrepancy, $d = 2$, identical seed across methods.

N	numpy	Sobol’	LHS (random-cd)
64	3.63×10^{-3}	1.46×10^{-4}	1.40×10^{-4}
128	3.33×10^{-3}	4.17×10^{-5}	3.75×10^{-5}
256	1.39×10^{-3}	1.11×10^{-5}	1.23×10^{-5}
512	7.64×10^{-4}	2.93×10^{-6}	3.82×10^{-6}

The same pattern holds at higher joint dimension: at $d = 4$ (the Class-B-partial per-reaction-batch dimensionality), $N = 64$, the discrepancies are 1.53×10^{-2} (numpy), 1.03×10^{-3} (Sobol’), and 1.13×10^{-3} (LHS) – roughly a $15\times$ improvement persists even as d grows.

4.2 Fallback/overflow rates under rejection sampling

To verify the row-reservation scheme behaves correctly under the actual rejection-sampling code (not a re-implemented stand-in), the modified `_psac_class_A`, `_psac_class_D_m2` (via `getClassB.partial`), `_psac_class_C`, and `_psac_fsac` (via `get_fsac.partial`) were exercised directly against a synthetic, well-conditioned 3×3 Cholesky factor, $N=64$ Sobol'-drawn rows, across $m = 1, 2, 3$ parameter subsets. Table 3 reports the resulting overflow rate (fraction of accepted samples whose first attempt was rejected, forcing a fallback draw) – the key diagnostic for how much rejection sampling erodes the joint low-discrepancy structure in practice.

Table 3: Overflow rate (first attempt rejected) by class and active-parameter count m , $N = 64$ Sobol'-seeded synthetic test.

Class	$m = 1$	$m = 2$	$m = 3$
A (partial)	0.172	0.000	0.000
B (partial) [†]	–	0.000	–
C (partial)	–	0.061	0.000
f-SAC (partial)	0.543	0.462	0.000

[†]Class-B's $m = 3$ case is routed to the SHGO-based `getB2Zeta` solve (no rejection step) rather than through `getClassB.partial`; Class-C is undefined for $m = 1$.

The overall pattern – most samples accept on the first attempt and therefore retain their designed shadow-row coordinates exactly, with only the minority that reject falling back to an independent stream – holds across every class tested. f-SAC's elevated overflow at $m = 1, 2$ reflects both its stricter sign-change acceptance criterion and, for $m = 1$, the documented recursive-delegation row-alignment looseness in §3.3; $m = 3$'s zero overflow for this particular synthetic matrix should not be read as a general guarantee, only as confirmation that the mechanism activates only when genuinely needed. In every case the routine still returned exactly the requested number of valid, full-space ζ vectors with no crashes or shape mismatches.

4.3 The random-shuffling step erodes joint structure

After the unshuffled block is built, the existing code (left unmodified here, per instruction) independently `np.random.shuffle`s each reaction's own samples four times to build the larger "shuffled" block. This preserves each reaction's one-dimensional marginal distribution exactly – every value that was drawn still appears somewhere – but independently permuting each reaction's row order destroys the deliberate cross-reaction pairing the joint shadow block was built to provide. A 2-reaction, $d = 4$ joint Sobol' block ($N = 64$) had centred L_2 discrepancy 1.05×10^{-3} before shuffling; after applying the same independent-per-reaction-permutation operation the shuffling step performs, discrepancy rose to 2.00×10^{-3} – roughly double, and in fact somewhat worse than a fresh LHS draw at the same N, d would give (1.02×10^{-3}), though still far better than plain pseudo-random at the same setting (9.34×10^{-3}). In short: shuffling does not destroy all of the benefit of switching to Sobol'/LHS, but it gives back a measurable fraction of it, and the shuffled block's joint structure ends up closer to (slightly worse than) what an independent LHS draw would have produced directly, regardless of which method seeded the pre-shuffle block. A designed permutation (e.g. an argsort of a fresh low-discrepancy draw, rather than `np.random.shuffle`) is a natural follow-up if the shuffled block's coverage matters as much as the unshuffled block's; building the full block jointly upfront without a shuffling step at all would avoid the issue entirely but at higher up-front sampling cost, which is presumably why the shuffling trick exists in the first place.

5 Practical recommendations

For a mechanism with J active reactions and a partial-parameter design, the joint Sobol’ dimensionality reaches $4J$ for Class-B (the largest of the four classes); `scipy`’s direction-number table supports up to $d = 21201$, so this is not a practical constraint until J exceeds roughly five thousand reactions. Below that, Sobol’ is the recommended default: it gives the same order-of-magnitude discrepancy improvement as LHS (Tables 2) while being faster to generate at the sample counts used here, at the cost of the power-of-two rounding (Table 3’s N values are themselves post-rounding). LHS is the better choice if the rounding-driven growth in sample count is undesirable for a specific run (e.g. a fixed external compute budget) or if a non-power-of-two count must match a previous run exactly for a side-by-side comparison. The legacy ‘`numpy`’ method remains available and is the right choice only for reproducing pre-existing results bit-for-bit.

6 Scope explicitly not covered by this pass

Two pieces of the existing codebase were identified but deliberately left untouched, for the user to revisit if desired:

1. `getSamples()` contains a second, older, and structurally different full-space f-SAC implementation (an inline `while len(temp)<1000` rejection sampler using `np.random.rand(3)` directly, not delegating to any `UncertaintyExtractor` method). This is unrelated to the `get_fSAC_partial/_psac_fsac` machinery treated above and was not given the joint-shadow treatment.
2. The two hard-coded assembly-loop bounds in `getSamples()` (`range(int(self.sim*0.2))` and `range(int(self.sim*0.8))`) were fixed to use the actual unshuffled/shuffled row counts rather than a fixed fraction of `self.sim` – otherwise Sobol’s power-of-two growth would have been silently truncated back down at exactly the point it was meant to be preserved. The partial path’s equivalent assembly already used the actual counts and needed no such fix.

7 Conclusion

The joint Sobol’/LHS shadow-matrix architecture delivers the requested one-to-two-order-of-magnitude discrepancy improvement over the legacy per-reaction pseudo-random design, for both the full and reduced-parameter (p-SAC) paths, including the explicitly requested f-SAC treatment, while preserving every existing rejection-sampling routine’s numerical behaviour through a reserved-row-per-sample-with-fallback scheme validated end-to-end against the real (not re-implemented) code. The one place this benefit is partially eroded – the existing random-shuffling step – has been measured rather than assumed, and the measurement is included here so the magnitude of the trade-off is visible rather than hidden.